Now I need the complete documentation how and what to do ? also what is the purpose of it and how it looks like in meraki ? also in meraki what I need to do? how can I have the complete SDA workflow in meraki ? from planning to implementation like what we have in SDA

Here is your **complete end-to-end documentation** — from planning on paper to a live LISP+VXLAN fabric running through Meraki, with no Catalyst Center.

Complete SDA Deployment via Meraki — Zero CC

What Is This and Why Are We Building It?

SD-Access is fundamentally just **LISP + VXLAN + Policy** running on IOS-XE switches. Catalyst Center is the *optional* orchestrator that configures these protocols — not a mandatory piece of the fabric itself. What you are building is a **Meraki Workflow** that replaces CC's guided setup wizard by pushing the same LISP and VXLAN YANG configuration directly to your switches via IOS-XE RESTCONF API. The end result is **identical to a CC-deployed SDA fabric**, just without the CC infrastructure dependency.

Phase Overview

Every phase has a clear purpose and a specific set of actions. Nothing overlaps.

PHASE 0 — Planning (Before Touching Anything)

This is the most important phase. SDA deployment without a controller requires you to pre-plan everything that CC would normally calculate for you automatically.

0.1 — Topology Design

Assign roles to your two switches:

Switch	Role	Why
C9500	Border Node + Control Plane Node	C9500 is the most powerful — it handles BGP L3 handoff to external network AND runs the LISP Map Server/Resolver brain

Switch	Role	Why
C9300	Fabric Edge Node	C9300 connects end devices — it encapsulates traffic into VXLAN tunnels toward C9500

0.2 — IP Address Schema

Plan all IPs before you start. Write these down:

```
MANAGEMENT NETWORK
  C9500 Mgmt IP:     192.168.1.100 /24
  C9300 Mgmt IP:     192.168.1.101 /24
  Relay Server IP:   192.168.1.50  /24
  Gateway:           192.168.1.1

UNDERLAY NETWORK (Loopback — VTEP source)
  C9500 Lo0:         10.255.255.1/32
  C9300 Lo0:         10.255.255.2/32
  OSPF Area:         0

OVERLAY NETWORK (User traffic inside VXLAN)
  CORP_VN Data:      192.168.10.0/24 → VLAN 100 → VNI 100
  CORP_VN Voice:     192.168.20.0/24 → VLAN 200 → VNI 200

LISP
  Instance ID:       100
  Map Server IP:     10.255.255.1 (C9500 Lo0)
  Map Resolver IP:   10.255.255.1 (same device for POC)

BGP (Border Handoff)
  C9500 Internal AS: 65001
  Upstream Router:   192.168.1.1
  Upstream AS:       65000
```

0.3 — Port Planning

```
C9500 TenGigabitEthernet1/0/1  → Uplink to router (BGP)
C9500 TenGigabitEthernet1/0/2  → VXLAN trunk to C9300
C9300 GigabitEthernet1/0/1     → Trunk to C9500
C9300 GigabitEthernet1/0/10    → Access port for end device (VLAN 100)
```

PHASE 1 — Switch Preparation (CLI — One Time Only)

This phase is done **once via CLI or Cloud CLI** in Meraki. After this, everything else is automated.

1.1 — C9500 Base Configuration

```
! ── Management ──────────────────────────────────
interface Vlan1
 ip address 192.168.1.100 255.255.255.0
 no shutdown

! ── Loopback (VTEP Source) ──────────────────────
interface Loopback0
 ip address 10.255.255.1 255.255.255.255
 no shutdown

! ── Enable RESTCONF ─────────────────────────────
restconf
ip http secure-server
ip http secure-ciphersuite HIGH
netconf-yang

! ── OSPF Underlay ───────────────────────────────
router ospf 1
 network 10.255.255.1 0.0.0.0 area 0
 network 192.168.1.0  0.0.0.255 area 0

write memory
```

1.2 — C9300 Base Configuration

```
! ── Management ──────────────────────────────────
interface Vlan1
 ip address 192.168.1.101 255.255.255.0
 no shutdown

! ── Loopback (VTEP Source) ──────────────────────
interface Loopback0
 ip address 10.255.255.2 255.255.255.255
 no shutdown

! ── Enable RESTCONF ─────────────────────────────
restconf
ip http secure-server
ip http secure-ciphersuite HIGH
netconf-yang

! ── OSPF Underlay ───────────────────────────────
router ospf 1
 network 10.255.255.2 0.0.0.0 area 0
 network 192.168.1.0  0.0.0.255 area 0

write memory
```

1.3 — Verify Everything Is Ready

On **both switches**, run:

```
show ip http secure server status
! Must show: HTTP Secure server status: Listen on port 443

show yang module | include lisp
! Must show: cisco-ios-xe-lisp

ping 10.255.255.2 source Loopback0
! C9500 must reach C9300 Loopback via OSPF
```

☁ PHASE 2 — Meraki Dashboard Setup

This is where you work entirely inside `dashboard.meraki.com`. Here is exactly what it looks like and where to click:

2.1 — Onboard Switches to Meraki (Cloud Monitoring Mode)

Where: `Organization → Devices → Add Devices`

1. Go to `Organization → Configure → Cloud Monitoring for Catalyst`
2. Click `Add Devices`
3. Enter C9500 and C9300 management IPs
4. Follow the onboarding wizard — push a small monitoring config to each switch
5. After 2–3 minutes both switches show as 🟢 `Online` under `Organization → Monitor → Devices`

What you see in Meraki after this:

- Device list showing C9500 and C9300 with hostname, IP, IOS XE version, uptime
- Topology map showing the link between C9500 and C9300
- Interface stats, CPU, memory — all visible in real-time

> **Important:** This puts switches in **Cloud Monitoring mode** — Meraki watches them, but does NOT manage their config. This is exactly what you want — your relay server + RESTCONF owns the config.

2.2 — Add the Relay Target

Where: `Organization → Automation → Workflows → Targets`

1. Click `Add Target`
2. Fill in:

```
Name:          SDA_Relay
Type:          HTTPS (or HTTP for local lab)
```

```
URL:            http://192.168.1.50:5000
Authentication: None (or Bearer Token for security)
```

3. Click `Test Connection`
4. Expected result: **"Connected — HTTP 200"**

> If using ngrok to expose relay to Meraki cloud:
> URL = `https://xxxx-xx-xxx-xx.ngrok.io`

2.3 — Import the Workflow

Where: `Organization → Automation → Workflows`

1. Click `Import Workflow`
2. Upload `meraki_workflow_export.json`
3. Workflow appears in library as `Deploy-SDA-Fabric-LISP-VXLAN`
4. Click it to open the **Workflow Canvas** — drag-and-drop visual editor showing:

```
[Input Form] → [HTTP POST to Relay] → [Condition] → [Success Email] / [Failure Email]
```

You can add, modify, or delete any activity by dragging from the left panel.

PHASE 3 — Deploy the Relay Server

Your relay server is a **50-line Python Flask app** running on a VM in your lab. It is the bridge between Meraki cloud and your switches.

3.1 — Run the Relay

On your lab server (Linux/Mac):

```
# Create working directory
mkdir sda-lab && cd sda-lab

# Copy all files (from previous responses) into this folder
# Install dependencies
pip3 install -r requirements.txt

# Edit .env — fill in your switch IPs and passwords
nano .env

# Start relay
python3 sda_relay_server.py
```

Expected output:

```
INFO - Starting SDA Relay Server — Port 5000
INFO - C9500: 192.168.1.100
```

```
INFO - C9300: 192.168.1.101
 * Running on http://0.0.0.0:5000
```

3.2 — Test Relay Connectivity to Switches

```
curl http://localhost:5000/health
# {"status": "relay_running", "timestamp": "2026-03-31T21:47:00"}

curl -X POST http://localhost:5000/webhook \
  -H "Content-Type: application/json" \
  -d '{"action":"health_check_devices"}'
# {"status":"success","c9500_reachable":true,"c9300_reachable":true}
```

Both must return `true`. If not, fix RESTCONF on the switch before proceeding.

3.3 — Expose Relay to Meraki Cloud (if needed)

If your relay is behind NAT:

```
# Install ngrok
brew install ngrok        # Mac
# OR download from https://ngrok.com/download

# Expose port 5000
ngrok http 5000

# Note the URL displayed:
# Forwarding → https://abc123.ngrok.io → localhost:5000

# Update Meraki Target URL to: https://abc123.ngrok.io
```

PHASE 4 — Execute the Workflow

This is where Meraki becomes your SD-Access deployment wizard.

Where: `Organization → Automation → Workflows → Deploy-SDA-Fabric-LISP-VXLAN → Execute`

4.1 — Fill in the Workflow Form

Meraki shows you a clean input form (like a wizard):

```
┌──────────────────────────────────────────┐
│  Deploy SD-Access Fabric                 │
│  ──────────────────────────────────────  │
│  Fabric Name:     [POC_SDA_Fabric      ]|
│  LISP Instance:   [100                 ]|
│  C9500 BGP AS:    [65001               ]|
│  Data VNI:        [100                 ]|
│  Data VLAN:       [100                 ]|
```

```
|                                          |
|          [▶ Execute Workflow]            |
└──────────────────────────────────────────┘
```

4.2 — What Happens Behind the Scenes

Click **Execute** and the following happens automatically:

```
Meraki Workflow fires
    ↓
POST /webhook → SDA_Relay (192.168.1.50:5000)
Body: {action: "deploy_lisp_vxlan", fabric_name: "POC_SDA_Fabric", ...}
    ↓
Relay Step 1: RESTCONF PUT → C9500
  Endpoint: /restconf/data/Cisco-IOS-XE-lisp:lisp
  Payload: LISP Map Server, Map Resolver, Instance 100
  Result: ✅ HTTP 204 — LISP configured
    ↓
Relay Step 2: RESTCONF PUT → C9500
  Endpoint: /restconf/data/Cisco-IOS-XE-nve:nve
  Payload: NVE1 source Lo0, VNI 100 → VLAN 100
  Result: ✅ HTTP 204 — VXLAN configured
    ↓
Relay Step 3: RESTCONF PUT → C9300
  Endpoint: /restconf/data/Cisco-IOS-XE-nve:nve
  Payload: NVE1 source Lo0, VNI 100 → VLAN 100
  Result: ✅ HTTP 204 — VXLAN configured
    ↓
Relay returns: {"status":"success","message":"Fabric deployment completed"}
    ↓
Meraki Workflow: ✅ Condition = success
    ↓
Meraki Email: "✅ SD-Access Fabric Deployed: POC_SDA_Fabric"
```

4.3 — Monitor in Meraki

Where: `Organization → Automation → Workflows → Execution History`

You see a real-time log of each activity:

```
✅ Input Form collected
✅ HTTP POST to SDA_Relay — 200 OK
✅ Condition check — passed
✅ Email notification — sent
Total Duration: 18 seconds
```

✅ PHASE 5 — Validation

5.1 — Validate from Meraki Cloud CLI

Where: `Organization → Monitor → Devices → C9500 → Tools → Cloud CLI`

Type directly in the browser terminal:

```
show lisp site
show lisp instance-id 100 statistics
show lisp map-cache
```

Expected output on C9500:

```
LISP Site Registration Information
* = Some locators are down or unreachable
# = Some registrations are sourced by reliable transport

Site Name      Last      Up     Who Last            Inst     EID Prefix
               Register         Registered          ID
site_uci       never     yes    --                  100      192.168.10.0/24
```

Switch to C9300 in Cloud CLI:

```
show vxlan detail
show interface nve 1
```

Expected output on C9300:

```
Interface NVE1 is up, line protocol is up
  Hardware is NVE
  Source-Interface: Loopback0 primary IP: 10.255.255.2
  Number of L2 VNIs: 1
  L2 VNI: 100  VLAN: 100  Type: LISP
```

5.2 — Run Automated Validation Script

```
python3 sda_deployment_validator.py
```

Expected output:

```
✅ RESTCONF Connectivity — C9500: PASSED
✅ RESTCONF Connectivity — C9300: PASSED
✅ LISP on C9500: PASSED
✅ VXLAN NVE — C9500: PASSED
✅ VXLAN NVE — C9300: PASSED
✅ VNI 100 → VLAN 100 Mapping: PASSED
```

```
──────────────────────────────────────────
RESULT: 6 passed, 0 failed
```

5.3 — End-to-End Connectivity Test

Connect a laptop to **C9300 GigabitEthernet1/0/10**. It should:

1. Receive an IP from 192.168.10.0/24 (DHCP from your server or static)
2. Be reachable from C9500 via the VXLAN tunnel
3. Show up as a LISP endpoint registration on C9500:

```
show lisp instance-id 100 database
```

What You Have vs What CC Provides — Comparison

Step	Catalyst Center	Your Meraki Workflow
Site design	GUI tree builder	IP schema on paper
IP Pool management	CC built-in IPAM	Pre-planned manually
Fabric creation	One-click guided wizard	Workflow form → relay
Auth profile	Dropdown (No Auth / ISE)	Hardcoded in relay (No Auth)
Virtual Network	GUI form	YANG payload via RESTCONF
Device role assignment	Drag device to role	Relay pushes LISP/VXLAN YANG
Port onboarding	GUI port picker	RESTCONF interface payload
Monitoring	CC Assurance dashboard	Meraki Dashboard + Cloud CLI
Validation	CC built-in checks	Your validator script
Day-N changes	CC templates	Update YANG payload + re-run

Troubleshooting Quick Reference

Symptom	Cause	Fix
Relay health check fails	Relay not running	`python3 sda_relay_server.py`
`c9500_reachable: false`	RESTCONF not enabled	`restconf` + `ip http secure-server` on switch
HTTP 422 from RESTCONF	YANG payload invalid	Check `Cisco-IOS-XE-lisp.yang` for required fields
Meraki Workflow times out	Relay not internet-reachable	Start ngrok, update Meraki Target URL

Symptom	Cause	Fix
`show lisp site` empty	LISP not applied to interface	Add `lisp mobility` under interface config
VXLAN tunnel down	OSPF underlay broken	`show ip ospf neighbor` — C9500 must see C9300

What Comes Next — Your Growth Path

```
TODAY (POC)
  └── Manual pre-config on switches
  └── Relay server on local VM
  └── Meraki Workflow with manual form input
  └── Single site

PHASE 2
  └── Pre-config via RESTCONF too (zero CLI)
  └── Relay behind HTTPS with API key auth
  └── Workflow triggered by schedule or alert

PHASE 3
  └── Store all configs in Git (YAML)
  └── GitHub webhook triggers Meraki Workflow
  └── Full GitOps — every change tracked

PHASE 4
  └── Multi-site: Relay at every branch
  └── Meraki Workflows loops across 50 sites
  └── Single dashboard deploys fabric everywhere
```

This is not a workaround. This is **Cisco's own infrastructure APIs used the way they were designed to be used** — openly, programmatically, and without mandatory dependency on any single controller.